

SmartAttend — AI-Powered Multi-Modal Attendance System

Comprehensive Technical Documentation, System Architecture, & Viva / Interview Q&A;

1. High-Level Project Overview & Problem Statement

SmartAttend is an advanced, production-ready multi-modal biometric attendance system built with Python, Streamlit, PyTorch, SpeechBrain, dlib, scikit-learn, and Supabase (PostgreSQL). It automates classroom roll-call by integrating **Visual Face Recognition** and **Acoustic Voice Identification**.

Key Problems Solved:

- **Time Inefficiency:** Eliminates manual roll-call overhead in large lecture halls.
- **Proxy Attendance Prevention:** Dual biometric modalities (Face + Voice) prevent students from fraudulently marking attendance for missing peers.
- **Human Error:** Replaces physical registers and paper records with real-time cloud data persistence.

2. System Architecture & Tech Stack

The application follows a decoupled multi-layer architecture separating presentation (Streamlit UI), machine learning inference pipelines (Face & Voice engines), and database persistence (Supabase PostgreSQL).

Component	Technology / Framework	Technical Purpose
Frontend UI	Streamlit (Python)	Interactive web app dashboards, camera feeds & modal dialogs
Cloud Database	Supabase (PostgreSQL)	Real-time cloud database storage & vector persistence
Face Pipeline	dlib + scikit-learn	68-point landmark alignment, 128D embedding & SVM classifier
Voice Pipeline	SpeechBrain + librosa + PyTorch	VAD audio splitting & ECAPA-TDNN speaker embedding extraction
Security	bcrypt	Salted password hashing for teacher account authentication
Utilities	segno + PIL + pandas	Dynamic QR code generation, image loading & log analytics

3. Database Schema Design (Supabase PostgreSQL)

SmartAttend uses five relational tables managed via `src/database/db.py` and configured in `src/database/config.py`:

Table Name	Key Columns & Types	Description
teachers	teacher_id (PK), username, password (hashed), name	Stores teacher credentials hashed securely with BCrypt.
students	student_id (PK), name, face_embedding (float[]), voice_embedding (float[])	Stores student profiles and 128D/speaker biometric embedding vectors.
subjects	subject_id (PK), subject_code (UNIQUE), name, section, teacher_id (FK)	Stores course metadata created by teachers.
subject_students	id (PK), subject_id (FK), student_id (FK)	Junction table mapping course enrollments.
attendance_logs	id (PK), subject_id (FK), student_id (FK), timestamp, is_present (bool)	Historical log records of attendance per session.

4. Artificial Intelligence & Deep Learning Pipelines

A. Face Recognition Pipeline (src/pipelines/face_pipeline.py)

- Face Detection:** `dlib.get_frontal_face_detector()` detects all face bounding boxes in an image using Histogram of Oriented Gradients (HOG) combined with a Linear SVM windowing detector.
- Facial Landmark Alignment:** `dlib.shape_predictor` locates 68 3D facial landmarks (eyes, nose, jawline) to crop and align faces into standard orientation.
- 128D Embedding Generation:** `dlib.face_recognition_model_v1` computes a continuous 128-dimensional deep feature vector $E \in \mathbb{R}^{128}$.
- SVM Classification:** A scikit-learn Linear Support Vector Classifier (SVC) is dynamically trained on all registered student embeddings.
- Euclidean Verification Check:** To eliminate false positives, the system calculates L2 Euclidean norm between trained and detected vectors:

$$d(E_{train}, E_{detected}) = \sqrt{(\sum [E_{train,i} - E_{detected,i}]^2)} \leq 0.45 \text{ (Resemblance Threshold)}$$

B. Voice Speaker Identification Pipeline (src/pipelines/voice_pipeline.py)

- Audio Preprocessing & Resampling:** Raw audio files (.wav/.mp3) are loaded via `librosa.load(..., sr=16000)` and resampled to 16kHz mono.
- Voice Activity Detection (VAD):** `librosa.effects.split(audio, top_db=30)` splits contiguous speech segments and trims background silence gaps.
- ECAPA-TDNN Model Inference:** SpeechBrain's pre-trained `spkrec-ecapa-voxceleb` neural network extracts a deep acoustic speaker embedding vector.
- Unit L2 Normalization:** Embedding vector is normalized: $v_{norm} = v / ||v||$.

5. **Cosine Similarity Matching:** Computes dot product similarity between stored student embeddings and candidate segment vectors:

$$\text{Similarity} = \mathbf{v}_{\text{candidate}} \cdot \mathbf{v}_{\text{stored}} = \sum (\mathbf{v}_{\text{candidate},i} \times \mathbf{v}_{\text{stored},i}) \geq 0.40$$

(Acceptance Threshold)

5. Step-by-Step Feature Implementation & Workflows

1. Student Webcam FaceID Login & Profile Setup:

- Students log in without typing passwords by capturing their face via webcam (`st.camera_input()`).
- If recognized, the system sets session variables and loads their student dashboard.
- Unrecognized faces trigger a profile creation form where the student provides their name and optional voice prompt.

2. Teacher Course Management & QR Code Sharing:

- Teachers register/login with BCrypt hashed credentials.
- Creating a subject generates a 6-character code (e.g., CS101), a deep link (`/?join-code=CS101`), and a PNG QR Code using segno.
- Students auto-enroll by scanning the QR code or entering the join code.

3. Taking AI Attendance & Manual Overrides:

- **Face Attendance:** Teacher uploads classroom group photos. The system scans all faces, predicts IDs using SVM + distance thresholds, and renders preview tables before logging.
- **Voice Attendance:** Teacher uploads class speech audio clips. The system performs VAD segmentation, matches speaker embeddings, and logs present students.
- **Manual Attendance:** Teachers can manually override student status with date/time selection controls.

6. Project Directory Map

File / Path	Function & Core Responsibilities
app.py	Main entry point, custom CSS theme injections, query param handler & route dispatcher.
src/database/db.py	Supabase SQL queries: CRUD operations for teachers, students, subjects, and logs.
src/database/config.py	Initializes Supabase Client using Streamlit secrets (SUPABASE_URL, SUPABASE_KEY).
src/pipelines/face_pipeline.py	Face detection, 128D vector extraction, SVM classifier training & Euclidean verification.
src/pipelines/voice_pipeline.py	SpeechBrain model loading, librosa VAD speech splitting & Cosine similarity identification.
src/screen/teacher_screen.py	Teacher portal UI layout: Auth views and 4-tab control dashboard.
src/screen/student_screen.py	Student portal UI layout: FaceID login, onboarding & subject attendance metrics.
src/components/	Streamlit modal dialogs (@st.dialog) for course creation, QR sharing, voice & manual attendance.

7. Comprehensive Viva & Interview Questions & Answers

Use these curated technical questions and answers to prepare for project presentations, viva examinations, or technical interviews:

Q1: What makes SmartAttend a multi-modal system, and why is this beneficial?

SmartAttend is multi-modal because it integrates both **Visual Biometrics (Face Recognition)** and **Acoustic Biometrics (Voice Speaker Identification)**.

Benefits: Multi-modality provides flexibility (e.g., fallback to voice attendance if lighting is poor or faces are covered with masks) and significantly increases security by preventing proxy attendance through redundant verification channels.

Q2: How does the Face Recognition pipeline convert a human face into mathematical data?

The pipeline consists of three sequential steps:

1. **Detection:** `dlib.get_frontal_face_detector()` locates the bounding box around a human face using HOG + Linear SVM.
 2. **Alignment:** `dlib.shape_predictor` locates 68 3D facial landmarks to scale and align the facial features.
 3. **Descriptor Extraction:** `dlib.face_recognition_model_v1` passes the cropped face through a deep ResNet neural network to generate a **128-dimensional floating-point vector** representing unique facial geometry.
-

Q3: How does the system detect and recognize multiple students in a single classroom photograph?

When a teacher uploads a group photo, `dlib` detects *all* bounding boxes in the photo. For each face, a 128D embedding vector is generated. The trained Linear Support Vector Classifier (SVC) predicts the candidate student ID for each vector. Next, an L2 Euclidean norm distance check (threshold ≤ 0.45) confirms the candidate match before marking attendance.

Q4: How does Voice Speaker Identification isolate individual speakers in a bulk audio recording?

The voice engine processes raw audio as follows:

1. **Resampling:** Audio is resampled to 16kHz mono via `librosa.load`.
 2. **Voice Activity Detection (VAD):** `librosa.effects.split(..., top_db=30)` removes background silences and isolates active speech segments.
 3. **Deep Embedding:** SpeechBrain's ECAPA-TDNN network computes a speaker embedding for each segment.
 4. **Cosine Matching:** Unit-normalized embeddings are compared against enrolled candidate vectors using dot product cosine similarity (threshold ≥ 0.40).
-

Q5: Why did you choose Linear Support Vector Classifier (SVC) instead of a basic nearest-neighbor search?

Linear SVC computes optimal separating hyperplanes between student embedding clusters in 128D space. It provides high classification speed ($O(d)$ dot products), handles multi-class student data cleanly, and scales better than raw k-NN. Combining SVM classification with an explicit Euclidean distance cutoff ensures high precision while preventing false matches.

Q6: How is database security handled for teacher authentication?

Teacher passwords are never stored as plain text. In `src/database/db.py`, passwords are encrypted using salted BCrypt hashing (`bcrypt.hashpw`). During authentication, `bcrypt.checkpw` verifies the presented credentials against the stored salt hash, preventing plain text credential theft and rainbow table attacks.

Q7: How does student auto-enrollment via URL or QR Code work technically?

When a teacher creates a course, a unique 6-character code (e.g. CS101) is generated. The system builds a URL (`/?join-code=CS101`) and renders a QR code using `segno`. When a student scans the QR code or clicks the link, `app.py` extracts `st.query_params.get('join-code')`, triggers the `auto_enroll_dialog`, and creates an entry in the `subject_students` Supabase table.

Q8: What happens if an unregistered stranger appears in a classroom photo?

The stranger will **not** be misidentified as an enrolled student. Although the SVM classifier will output a candidate prediction, the calculated Euclidean distance $||E_{train} - E_{detected}||$ will exceed the strict 0.45 resemblance threshold. Consequently, the match is rejected.

Q9: How are attendance records aggregated and displayed in analytics views?

Attendance logs are stored as timestamped ISO records in Supabase. The Teacher Records view uses `pandas` to group logs by subject and timestamp (`df.groupby(['ts_group', 'Subject'])`) to aggregate total present count versus total enrolled count, producing concise summary statistics.

Q10: What are the main system limitations and potential future enhancements?

Limitations: Face recognition requires adequate frontal lighting and minimal face occlusion; voice recognition performance degrades in high ambient noise environment.

Future Enhancements: Implementing real-time WebRTC video streams, anti-spoofing liveness detection (blink/head motion checks), and integrating deep noise suppression algorithms (e.g. `DeepFilterNet`) before VAD processing.
